
SHELF CARE — FULL CODING FLOW JupyterLab + Gradio + Hugging Face Spaces

PHASE 1 — Environment Setup (Week 1)

1.1 Install your libraries

Run this as your very first notebook cell. Install everything you need for the whole project.

None

```
!pip install gradio anthropic spotipy requests python-dotenv
Pillow
```

1.2 Set up your API keys safely

Never hardcode keys in your notebook. Create a file called `.env` in the same folder as your notebook and put your keys there. Load them in Python like this:

None

```
# .env file:
ANTHROPIC_API_KEY=your_key_here
SPOTIFY_CLIENT_ID=your_id_here
SPOTIFY_CLIENT_SECRET=your_secret_here
TMDB_API_KEY=your_key_here
UNSPLASH_ACCESS_KEY=your_key_here

# In your notebook:
from dotenv import load_dotenv
import os
load_dotenv()
ANTHROPIC_KEY = os.getenv("ANTHROPIC_API_KEY")
```

1.3 Build your place database

Create a file called `places.json` with around 30 locations. Each one needs a name, country, mood tags, and a short vibe description. Make sure you have geographic and mood diversity — not just European cities.

None

```
[
  {
    "name": "Stockholm",
    "country": "Sweden",
    "region": "Northern Europe",
    "mood_tags": ["dark", "cold", "psychological", "minimal",
"Nordic"],
    "vibe": "Icy precision with something unsettling
underneath"
  },
  {
    "name": "Bali",
    "country": "Indonesia",
    "region": "Southeast Asia",
    "mood_tags": ["warm", "spiritual", "lush", "healing",
"tropical"],
    "vibe": "Sun-soaked and ceremonial, quietly rebuilding
you"
  }
]
```

1.4 Set up your folder structure

None

```
shelfcare/
  notebook.ipynb      # main working notebook
  app.py              # Gradio app file for Hugging Face
  places.json         # your location database
  requirements.txt    # libraries list for deployment
```

```
.env                # API keys – never share or commit
this
utils/
    claude_calls.py  # your 3 Claude prompt functions
    api_calls.py     # TMDB, Spotify, Unsplash, Open
Library
```

PHASE 2 — Claude API: The Three Prompts (Week 1-2)

2.1 Prompt 1 — Vibe Extraction

This is the most important prompt. Test it alone with 10+ books before touching anything else. If the JSON output is good, everything downstream works.

```
None
import anthropic
import json

client =
anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def extract_vibe(book_title, mood, pace, setting, depth):
    prompt = f"""
    You are a literary analyst. Given the book "{book_title}",
    return ONLY a JSON object with these exact fields:
    - tone: short phrase e.g. "dark and psychological"
    - mood_tags: list of exactly 5 keywords
    - setting_type: e.g. "urban", "wilderness", "suburban"
    - pacing: e.g. "slow burn", "fast-paced", "steady"
    - emotional_core: one sentence describing the central
    emotion

    User mood preference context:
```

```
Dark/Light: {mood}/100, Slow/Fast: {pace}/100,  
Urban/Wild: {setting}/100, Surface/Intense: {depth}/100
```

```
Return valid JSON only. No explanation, no markdown.  
"""
```

```
response = client.messages.create(  
    model="claude-sonnet-4-5",  
    max_tokens=500,  
    messages=[{"role": "user", "content": prompt}]  
)  
return json.loads(response.content[0].text)
```

2.2 Prompt 2 — Place Matching

Takes the JSON from Prompt 1 and your places.json and asks Claude to pick the best match.

None

```
def match_place(vibe_json, places_list):  
    prompt = f"""  
    Given this book vibe profile:  
    {json.dumps(vibe_json, indent=2)}  
  
    And this list of available places:  
    {json.dumps(places_list, indent=2)}  
  
    Pick the single best matching place based on mood,  
    atmosphere,  
    and emotional resonance. Return ONLY this JSON:  
    {{"place_name": "...", "country": "...", "reason": "one  
sentence"}}  
    """  
  
    response = client.messages.create(  
        model="claude-sonnet-4-5",  
        max_tokens=200,
```

```

        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

```

2.3 Prompt 3 — Vibe Summary (Why This Matches)

This is your biggest differentiator. Spend real time getting this prompt right — "both are dark" is not good enough.

None

```

def generate_vibe_summary(book_title, place_name):
    prompt = f"""
    Write a 2-3 sentence explanation of why someone who just
    finished
    "{book_title}" would feel emotionally at home in
    {place_name}.

    Rules:
    - Be specific about atmosphere, sensory details, and
    feeling
    - Do not mention genre or plot
    - Do not be generic or obvious
    - Write as if describing a feeling, not summarizing a book
    - Make it feel like a friend wrote it, not a critic
    """

    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text.strip()

```

2.4 Test all 3 prompts chained together

Do not move to Phase 3 until these outputs consistently feel right.

None

```
with open("places.json") as f:
    places = json.load(f)

test_books = [
    "Gone Girl", "Eat Pray Love", "Dune",
    "The Great Gatsby", "Memoirs of a Geisha",
    "Into the Wild", "Big Little Lies",
    "Crazy Rich Asians", "The Alchemist", "1984"
]

for book in test_books:
    vibe = extract_vibe(book, 50, 50, 50, 50)
    place = match_place(vibe, places)
    summary = generate_vibe_summary(book, place["place_name"])
    print(f"{book} -> {place['place_name']}")
    print(summary)
    print("---")
```

PHASE 3 — API Integrations: One at a Time (Week 2)

3.1 Open Library — Book cover and description

Free, no key needed. Build and fully test this before moving to the next API.

None

```
import requests

def get_book_info(title):
    url = "https://openlibrary.org/search.json"
    params = {"title": title, "limit": 1}
    r = requests.get(url, params=params).json()
    if not r["docs"]:
        return None
```

```

doc = r["docs"][0]
cover_id = doc.get("cover_i")
cover_url =
f"https://covers.openlibrary.org/b/id/{cover_id}-L.jpg"
return {
    "title": doc.get("title"),
    "author": doc.get("author_name", ["Unknown"])[0],
    "cover_url": cover_url if cover_id else None,
    "description": doc.get("first_sentence", [""])[0]
}

```

3.2 TMDB — Movie recommendation

Search by place name and mood tags. Filter out direct adaptations of the same book title.

None

```

def get_movie(place_name, mood_tags, book_title):
    base = "https://api.themoviedb.org/3"
    query = f"{place_name} {' '.join(mood_tags[:2])}"
    r = requests.get(f"{base}/search/movie", params={
        "api_key": os.getenv("TMDB_API_KEY"),
        "query": query,
        "language": "en-US"
    }).json()

    for movie in r.get("results", []):
        # Filter out direct book adaptations
        if book_title.lower() not in movie["title"].lower():
            poster =
f"https://image.tmdb.org/t/p/w500{movie['poster_path']}"
            return {
                "title": movie["title"],
                "overview": movie["overview"],
                "poster_url": poster
            }

```

```
    }  
    return None
```

3.3 Spotify — Mood playlist

Use Spotipy with Client Credentials flow — simpler than full OAuth since you are not accessing personal user data.

```
None  
import spotipy  
from spotipy.oauth2 import SpotifyClientCredentials  
  
sp = spotipy.Spotify(auth_manager=SpotifyClientCredentials(  
    client_id=os.getenv("SPOTIFY_CLIENT_ID"),  
    client_secret=os.getenv("SPOTIFY_CLIENT_SECRET")  
))  
  
def get_playlist(place_name, mood_tags):  
    query = f"{place_name} {mood_tags[0]} atmosphere"  
    results = sp.search(q=query, type="playlist", limit=1)  
    items = results["playlists"]["items"]  
    if not items:  
        return None  
    pl = items[0]  
    return {  
        "name": pl["name"],  
        "url": pl["external_urls"]["spotify"],  
        "image": pl["images"][0]["url"] if pl["images"] else  
None  
    }
```

3.4 Unsplash — Place banner image

Simplest of all. Grab the first result's full-size photo URL for your dashboard banner.

None

```
def get_place_image(place_name):
    r = requests.get(
        "https://api.unsplash.com/search/photos",
        params={
            "query": place_name,
            "per_page": 1,
            "orientation": "landscape"
        },
        headers={"Authorization": f"Client-ID
{os.getenv('UNSPLASH_ACCESS_KEY')}"})
    ).json()
    if not r["results"]:
        return None
    return r["results"][0]["urls"]["regular"]
```

PHASE 4 — Gradio UI: Building the Dashboard (Week 3)

4.1 Build the input section

None

```
import gradio as gr

with gr.Blocks(title="ShelfCare") as app:
    gr.Markdown("# ShelfCare\n*read, feel, discover*")

    with gr.Row():
        book_input = gr.Textbox(
            label="Book title",
            placeholder='e.g. "Gone Girl"'
        )

    with gr.Row():
```

```

        mood      = gr.Slider(0, 100, 50, label="Mood: Dark to
Light")
        pace      = gr.Slider(0, 100, 50, label="Pace: Slow to
Fast")
        setting   = gr.Slider(0, 100, 50, label="Setting: Urban
to Wild")
        depth     = gr.Slider(0, 100, 80, label="Depth: Surface
to Intense")

        submit_btn = gr.Button("Find my world")
        output     = gr.HTML()

```

4.2 Write the main backend function

One function that runs the full pipeline and returns everything to Gradio.

None

```

def generate(book, mood, pace, setting, depth):
    # Step 1: Claude vibe extraction
    vibe = extract_vibe(book, mood, pace, setting, depth)

    # Step 2: Claude place matching
    place_result = match_place(vibe, places)
    place_name = place_result["place_name"]

    # Step 3: Claude vibe summary
    summary = generate_vibe_summary(book, place_name)

    # Step 4: All four API calls
    book_info = get_book_info(book)
    movie     = get_movie(place_name, vibe["mood_tags"],
book)
    playlist  = get_playlist(place_name, vibe["mood_tags"])
    place_img = get_place_image(place_name)

```

```

# Step 5: Build and return HTML output card
return build_output_html(
    place_name, summary, book_info, movie, playlist,
    place_img
)

```

4.3 Build the output HTML card

None

```

def build_output_html(place, summary, book_info, movie,
    playlist, img_url):
    html = f"""
    <div style="font-family:Georgia; max-width:700px; margin:0
    auto">

        <!-- Place banner -->
        <div style="background-image:url({img_url});
        background-size:cover;
            background-position:center; height:180px;
        border-radius:12px;
            display:flex; align-items:flex-end; padding:16px;
        margin-bottom:16px">
            <h2 style="color:white; margin:0; text-shadow:0 1px
        4px rgba(0,0,0,0.6)">
                {place}
            </h2>
        </div>

        <!-- Vibe summary -->
        <p style="color:#4E7358; font-style:italic;
        font-size:15px;
            line-height:1.7; margin-bottom:20px">{summary}</p>
    """

```

```

        <!-- Movie and music cards -->
        <div style="display:grid; grid-template-columns:1fr 1fr;
gap:12px;
        margin-bottom:16px">
            <div style="background:#F4EDE0; padding:14px;
border-radius:10px">
                <p style="font-size:11px; color:#9E8470; margin:0 0
4px">Watch next</p>
                <p style="font-weight:bold; margin:0 0
6px">{movie['title']}</p>
                <p style="font-size:12px; color:#5F5E5A; margin:0">
                    {movie['overview'][:120]}...
                </p>
            </div>
            <div style="background:#F4EDE0; padding:14px;
border-radius:10px">
                <p style="font-size:11px; color:#9E8470; margin:0 0
4px">Listen to</p>
                <a href="{playlist['url']}" style="font-weight:bold;
color:#4E7358">
                    {playlist['name']}
                </a>
            </div>
        </div>
    """
    return html

```

4.4 Connect the function and launch

None

```
submit_btn.click(
    fn=generate,
    inputs=[book_input, mood, pace, setting, depth],
    outputs=output
)

app.launch(share=True) # share=True gives a 72hr public link
```

PHASE 5 — Testing, Polish, and Deployment (Week 3-4)

5.1 Test with 50 books

None

```
test_books_extended = [
    # Thriller
    "Gone Girl", "The Girl with the Dragon Tattoo", "Big
    Little Lies",
    # Literary fiction
    "The Great Gatsby", "Normal People", "Pachinko",
    # Romance
    "Outlander", "The Notebook", "Beach Read",
    # Sci-fi / Fantasy
    "Dune", "The Handmaid's Tale", "Ender's Game",
    # Travel / Memoir
    "Eat Pray Love", "Into the Wild", "Wild",
    # Nonfiction
    "Sapiens", "The Body", "Educated",
    # Classics
    "Pride and Prejudice", "Anna Karenina", "1984"
    # keep adding until you hit 50
]
```

5.2 Add error handling to every API call

None

```
def safe_get_movie(place_name, mood_tags, book_title):
    try:
        result = get_movie(place_name, mood_tags, book_title)
        return result if result else {
            "title": "No match found",
            "overview": "Try a different book.",
            "poster_url": None
        }
    except Exception as e:
        print(f"TMDB error: {e}")
        return {"title": "Unavailable", "overview": "",
            "poster_url": None}

# Apply the same try/except pattern to all four API functions
```

5.3 Create requirements.txt

None

```
gradio
anthropic
spotipy
requests
python-dotenv
Pillow
```

5.4 Deploy to Hugging Face Spaces

1. Go to huggingface.co and create a free account
2. Click New Space, select Gradio as the SDK, name it shelfcare
3. In their web editor paste your app.py and requirements.txt
4. Go to Settings → Secrets and add all your API keys there
5. Hit Save — your app is live at huggingface.co/spaces/yourname/shelfcare

Your app.py on Hugging Face must end with just:

None

```
app.launch()
```

No `share=True` needed — Hugging Face handles the public URL automatically.

5.5 Pick your 5 demo books

None

```
DEMO_BOOKS = [  
    "Gone Girl",          # dark psychological thriller  
    "Eat Pray Love",      # warm spiritual travel  
    "Dune",               # epic desert myth  
    "The Great Gatsby",   # glamour and longing  
    "Memoirs of a Geisha" # quiet beauty and sacrifice  
]  
# Run these before demo day and screenshot every output as a  
# backup
```
